CO-1 LAB EXPERIMENTS

1. 8-PUZZLE PROBLEM

CODE:

```
from collections import deque
goal = ((1, 2, 3),
        (4, 5, 6),
        (7, 8, 0))
# Generate all possible moves
def get_neighbors(state):
    board = [list(row) for row in state]
    # Find blank tile (0)
    for i in range(3):
        for j in range(3):
            if board[i][j] == 0:
                x, y = i, j
    directions = [(-1,0), (1,0), (0,-1), (0,1)]
    neighbors = []
    for dx, dy in directions:
        nx = x + dx
        ny = y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            temp = [row[:] for row in board]
            temp[x][y], temp[nx][ny] = temp[nx][ny], temp[x][y]
            neighbors.append(tuple(tuple(row) for row in temp))
    return neighbors
# BFS Algorithm
def bfs(start):
    queue = deque()
    queue.append(start)
```

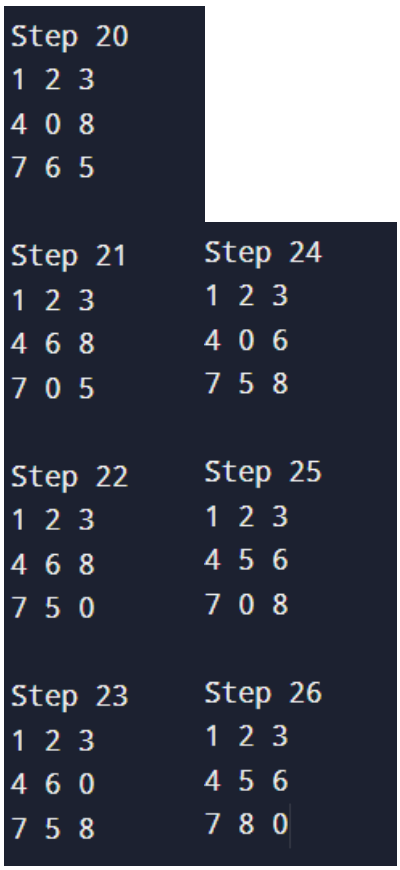
```
    visited = set()
    visited.add(start)
    parent = {}
    parent[start] = None
    while queue:
        current = queue.popleft()
        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]
            path.reverse()
            return path
        for neighbor in get_neighbors(current):
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = current
                queue.append(neighbor)
    return None
# ------------ Main Program ------------
print("Enter Initial State (3 rows):")
start = []
for i in range(3):
    row = tuple(map(int, input().split()))
    start.append(row)
start = tuple(start)
solution = bfs(start)
if solution:
    print("\nSolution Found!\n")
    for step, state in enumerate(solution):
        print("Step", step)
        for row in state:
```

```
            print(*row)
        print()
else:
    print("No Solution")
```

OUTPUT

```
Enter Initial State (3 rows):
3 4 2
6 7 1
0 8 5
```

```
Solution Found!

Step 0
3 4 2
6 7 1
0 8 5

Step 1
3 4 2
0 7 1
6 8 5

Step 2
3 4 2
7 0 1
6 8 5

Step 3
3 4 2
7 8 1
6 0 5

Step 4
3 4 2
7 8 1
0 6 5

Step 5
3 4 2
0 8 1
7 6 5

Step 6
0 4 2
3 8 1
7 6 5

Step 7
4 0 2
3 8 1
7 6 5

Step 8
4 8 2
3 0 1
7 6 5

Step 9
4 8 2
3 1 0
7 6 5

Step 10
4 8 0
3 1 2
7 6 5

Step 11
4 0 8
3 1 2
7 6 5

Step 12
4 1 8
3 0 2
7 6 5

Step 13
4 1 8
0 3 2
7 6 5

Step 14
0 1 8
4 3 2
7 6 5

Step 15
1 0 8
4 3 2
7 6 5

Step 16
1 3 8
4 0 2
7 6 5

Step 17
1 3 8
4 2 0
7 6 5

Step 18
1 3 0
4 2 8
7 6 5

Step 19
1 0 3
4 2 8
7 6 5
```



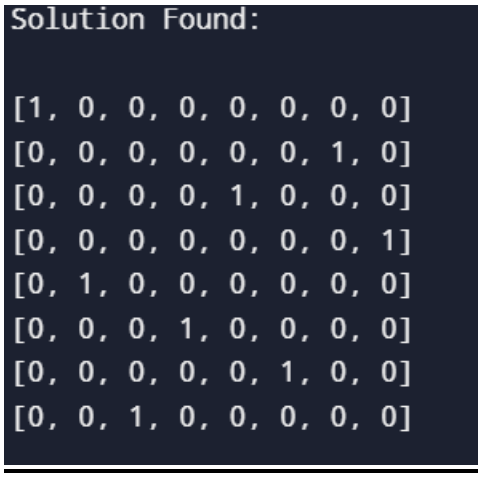
2. **8-QUEEN PROBLEM**

CODE:

```
N = 8
board = [[0] * N for _ in range(N)]
def is_safe(row, col):
    for i in range(col):
        if board[row][i]:
            return False
    i = row
    j = col
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1
```

```
    i = row
    j = col
    while i < N and j >= 0:
        if board[i][j]:
            return False
        i += 1
        j -= 1
    return True
def solve(col):
    if col == N:
        return True
    for row in range(N):
        if is_safe(row, col):
            board[row][col] = 1
            if solve(col + 1):
                return True
            board[row][col] = 0
    return False
if solve(0):
    print("Solution Found:\n")
    for row in board:
        print(row)
else:
    print("No Solution")
```

OUTPUT



3. **WATER JUG PROBLEM**

CODE:

```
from collections import deque
def water_jug():
    visited = set()
    queue = deque()
    # Start with both jugs empty
    queue.append((0, 0, []))
    while queue:
        a, b, path = queue.popleft()
        if (a, b) in visited:
            continue
        visited.add((a, b))
        path = path + [(a, b)]
        # Goal condition: exactly (2,0)
        if (a, b) == (2, 0):
            return path
        # Carefully ordered moves to match desired output
        next_states = [
            (4, b),  # Fill 4L jug
```

```
            (a, 3) # Fill 3L jug
            (0, b) # Empty 4L jug
            (a, 0) # Empty 3L jug
            (a - min (a, 3 - b), b + min (a, 3 - b)) # Pour A -> B
            (a + min (b, 4 - a), b - min (b, 4 - a)) # Pour B -> A
        ]
        for state in next_states:
            if state not in visited:
                queue.append((state[0], state[1], path))
result = water_jug()
print("Steps:")
for step in result:
    print(step)
```

OUTPUT

```
Steps:
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
(0, 2)
(2, 0)
```

4. **CRYPT-ARITHMETIC PROBLEM**

CODE:

```
from itertools import permutations
letters = ('S','E','N','D','M','O','R','Y')
digits = (0,1,2,3,4,5,6,7,8,9)
for p in permutations (digits, 8):
    S,E,N,D,M,O,R,Y = p
    if S == 0 or M == 0:
        continue
```

```
    SEND = S*1000 + E*100 + N*10 + D
    MORE = M*1000 + O*100 + R*10 + E
    MONEY = M*10000 + O*1000 + N*100 + E*10 + Y
    if SEND + MORE == MONEY:
        print("SEND =", SEND)
        print("MORE =", MORE)
        print("MONEY =", MONEY)
        break
```

OUTPUT

```
SEND = 9567
MORE = 1085
MONEY = 10652
```

5. MISSIONARIES CANNIBAL PROBLEM

CODE:

```
from collections import deque
start = (3, 3, 'L')
goal = (0, 0, 'R')
def is_valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    if m > 0 and m < c:  # missionaries outnumbered on left
        return False
    if (3 - m) > 0 and (3 - m) < (3 - c):  # missionaries outnumbered on right
        return False
    return True
def next_states(state):
    m, c, side = state
    moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
    states = []
```

```
    for dm, dc in moves:
        if side == 'L':
            nm = m - dm
            nc = c - dc
            ns = 'R'
        else:
            nm = m + dm
            nc = c + dc
            ns = 'L'
        if is_valid(nm, nc):
            states.append(((nm, nc, ns), (dm, dc)))  # include boat passengers
    return states
def bfs():
    q = deque([(start,[start],[] )])  # (state, path, moves)
    visited = set()
    while q:
        state, path, moves = q.popleft()
        if state == goal:
            return path, moves
        if state in visited:
            continue
        visited.add(state)
        for s, move in next_states(state):
            q.append((s, path+[s], moves+[move]))
solution, boat_moves = bfs()
print("Solution:")
for i, step in enumerate(solution):
    m, c, side = step
    left = (m, c)
    right = (3 - m, 3 - c)
    if i == 0:
        print(f"Step {i}: Left={left}, Right={right}, Boat={side}")
    else:
        dm, dc = boat_moves[i-1]
```

```
        print(f"Step {i}: Left={left}, Right={right}, Boat={side}, "
              f"Boat carried {dm}M {dc}C")
```

OUTPUT

```
Solution:
Step 0: Left=(3, 3), Right=(0, 0), Boat=L
Step 1: Left=(3, 1), Right=(0, 2), Boat=R, Boat carried 0M 2C
Step 2: Left=(3, 2), Right=(0, 1), Boat=L, Boat carried 0M 1C
Step 3: Left=(3, 0), Right=(0, 3), Boat=R, Boat carried 0M 2C
Step 4: Left=(3, 1), Right=(0, 2), Boat=L, Boat carried 0M 1C
Step 5: Left=(1, 1), Right=(2, 2), Boat=R, Boat carried 2M 0C
Step 6: Left=(2, 2), Right=(1, 1), Boat=L, Boat carried 1M 1C
Step 7: Left=(0, 2), Right=(3, 1), Boat=R, Boat carried 2M 0C
Step 8: Left=(0, 3), Right=(3, 0), Boat=L, Boat carried 0M 1C
Step 9: Left=(0, 1), Right=(3, 2), Boat=R, Boat carried 0M 2C
Step 10: Left=(1, 1), Right=(2, 2), Boat=L, Boat carried 1M 0C
Step 11: Left=(0, 0), Right=(3, 3), Boat=R, Boat carried 1M 1C
```

6. VACUUM CLEANER PROBLEM

CODE:

```
rooms = {'A': 'Dirty', 'B': 'Dirty'}   # initial environment
location = 'A'                         # vacuum starts in room A
step = 1
while True:
    print(f"\nStep {step}:")
    print("Vacuum Location:", location)
    print("Room Status:", rooms)
    # Action: Clean if dirty
    if rooms[location] == "Dirty":
        print(f"Action: Suck dirt in Room {location}")
        rooms[location] = "Clean"
    else:
        print(f"Action: Room {location} already clean")
    # Check goal condition
    if rooms['A'] == "Clean" and rooms['B'] == "Clean":
```

```
        print("\nAll Rooms Clean ✅")
        break
    # Action: Move to the other room
    if location == 'A':
        print("Action: Move Right → Room B")
        location = 'B'
    else:
        print("Action: Move Left → Room A")
        location = 'A'
    step += 1
```

OUTPUT

```
Step 1:
Vacuum Location: A
Room Status: {'A': 'Dirty', 'B': 'Dirty'}
Action: Suck dirt in Room A
Action: Move Right → Room B

Step 2:
Vacuum Location: B
Room Status: {'A': 'Clean', 'B': 'Dirty'}
Action: Suck dirt in Room B

All Rooms Clean ✅
```

7. BFS IMPLEMENTATION

CODE:

```
graph = {
'A':['B','C'],
'B':['D','E'],
'C':['F'],
'D':[],
'E':['F'],
'F':[]
```

```
}
visited = []
queue = []
start = 'A'
visited.append(start)
queue.append(start)
print("BFS Traversal:")
while queue:
    node = queue.pop(0)
    print(node,end=" ")
    for neighbour in graph[node]:
        if neighbour not in visited:
            visited.append(neighbour)
            queue.append(neighbour)
```

OUTPUT

```
BFS Traversal:
A B C D E F
```

8. DFS IMPLEMENTATION

CODE:

```
graph = {
'A':['B','C'],
'B':['D','E'],
'C':['F'],
'D':[],
'E':['F'],
'F':[]
}
```

```
visited = set()

def dfs(node):

    if node not in visited:

        print(node,end=" ")

        visited.add(node)

        for neighbour in graph[node]:

            dfs(neighbour)

print("DFS Traversal:")

dfs('A')
```

OUTPUT

```
DFS Traversal:
A B D E F C
```

9. TRAVELLING SALESMAN PROBLEM

CODE:

```
from itertools import permutations

graph = [
[0,10,15,20],
[10,0,35,25],
[15,35,0,30],
[20,25,30,0]
]

cities = [1,2,3]

min_cost = 9999

best_path = ()

for path in permutations(cities):

    cost = 0

    k = 0

    for city in path:
```

```
            cost += graph[k][city]
            k = city
        cost += graph[k][0]
        if cost < min_cost:
            min_cost = cost
            best_path = path
    print("Minimum Cost =",min_cost)
    print("Path = 0",end=" ")
    for city in best_path:
        print("->",city,end=" ")
    print("-> 0")
```

OUTPUT

```
Minimum Cost = 80
Path = 0 -> 1 -> 3 -> 2 -> 0
```

10. A* ALGORITHM

CODE:

```
graph = {
'A':[('B',1),('C',3)],
'B':[('D',3),('E',6)],
'C':[('F',5)],
'D':[],
'E':[('G',2)],
'F':[('G',2)],
'G':[]
}
```

```
heuristic = {
'A':7,
'B':6,
'C':4,
'D':3,
'E':2,
'F':1,
'G':0
}
open_list = [('A',0)]
visited = []
while open_list:
    open_list.sort(key=lambda x:x[1]+heuristic[x[0]])
    node,cost = open_list.pop(0)
    if node in visited:
        continue
    print(node,end=" ")
    visited.append(node)
    if node=='G':
        print("\nGoal Reached")
        break
    for neighbour,value in graph[node]:
        if neighbour not in visited:
            open_list.append((neighbour,cost+value))
```

OUTPUT

```
A B C D E F G
Goal Reached
```